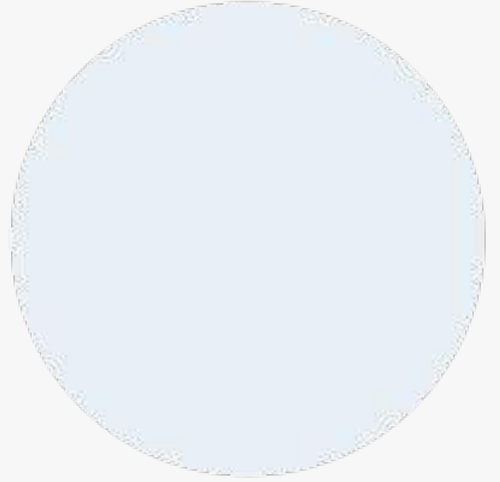


ASTON.



Лямбда и Stream API



astondevs.ru

Функциональные интерфейсы



Лямбда-выражения были добавлены в Java 8. Их основная цель – повысить читабельность и уменьшить количество кода. Но, прежде чем перейти к лямбдам, нам необходимо понимать функциональные интерфейсы.

Если интерфейс в Java содержит один и только один абстрактный метод, то он называется функциональным. Этот единственный метод определяет назначение интерфейса. Например, интерфейс `Runnable` из пакета `java.lang` является функциональным, потому, что он содержит только один метод `run()`.

Именно интерфейсы с одним единственным методом и подходят для использования лямбда-выражениями. Лямбда-выражение — это метод, завернутый в объект. И когда мы передаем куда-то такой объект — мы, по сути, передаем этот один единственный метод. Получается, нам не важно, как этот метод называется. Все, что нам важно — это параметры, которые этот метод принимает, и, собственно, сам код метода.

Объявление функционального интерфейса в Java



```
import java.lang.FunctionalInterface;  
@FunctionalInterface  
public interface MyInterface{  
    // один абстрактный метод  
    double getValue();  
}
```

В приведенном выше примере, интерфейс MyInterface имеет только один абстрактный метод getValue(). Значит, этот интерфейс — функциональный. Здесь мы использовали аннотацию @FunctionalInterface, которая помогает понять компилятору, что интерфейс функциональный. Следовательно, не позволяет иметь более одного абстрактного метода. Тем не менее, мы можем её опустить.

Лямбда-выражение



Лямбда-выражения пришли в Java из функционального программирования, а туда — из математики.

Лямбда-выражение — это такая функция. Можно считать, что это обычный метод в Java, только его особенность в том, что его можно передавать в другие методы в качестве аргумента. По сути, это реализация функционального интерфейса. Где видим интерфейс с одним методом — значит, такой анонимный класс можем переписать через лямбду. Если в интерфейсе больше/меньше одного метода — тогда нам лямбда-выражение не подойдет, и будем использовать анонимный класс, или даже обычный.

Основу лямбда-выражения составляет лямбда-оператор, который представляет стрелку $\rightarrow$. Этот оператор разделяет лямбда-выражение на две части: левая часть содержит список параметров выражения, а правая собственно представляет тело лямбда-выражения, где выполняются все действия.

Лямбда-выражение



Лямбда-выражение не выполняется само по себе, а образует реализацию метода, определенного в функциональном интерфейсе.

```
public class LambdaApp {  
  
    public static void main(String[] args) {  
  
        Operationable operation;  
        operation = (x,y)->x+y;  
  
        int result = operation.calculate(10, 20);  
        System.out.println(result); //30  
    }  
}  
  
interface Operationable{  
    int calculate(int x, int y);  
}
```


Лямбда-выражение



В роли функционального интерфейса выступает интерфейс Operationable, в котором определен один метод без реализации - метод calculate. Данный метод принимает два параметра - целых числа, и возвращает некоторое целое число.

По факту лямбда-выражения являются в некотором роде сокращенной формой внутренних анонимных классов, которые ранее применялись в Java.

В частности, предыдущий пример мы можем переписать следующим образом:

```
public class LambdaApp {  
  
    public static void main(String[] args) {  
  
        Operationable op = new Operationable() {  
  
            public int calculate(int x, int y) {  
  
                return x + y;  
            }  
        };  
        int z = op.calculate(20, 10);  
        System.out.println(z); // 30  
    }  
}  
  
interface Operationable {  
    int calculate(int x, int y);  
}
```

Работа с коллекциями



Чтобы объявить и использовать лямбда-выражение, основная программа разбивается на ряд этапов:

Определение ссылки на функциональный интерфейс:

```
Operationable operation;
```

Создание лямбда-выражения:

```
operation = (x, y) -> x + y;
```

Причем параметры лямбда-выражения соответствуют параметрам единственного метода интерфейса `Operationable`, а результат соответствует возвращаемому результату метода интерфейса. При этом нам не надо использовать ключевое слово `return` для возврата результата из лямбда-выражения.

Так, в методе интерфейса оба параметра представляют тип `int`, значит, в теле лямбда-выражения мы можем применить к ним сложение. Результат сложения также представляет тип `int`, объект которого возвращается методом интерфейса.

Использование лямбда-выражения в виде вызова метода интерфейса



```
int result = operation.calculate(10, 20);
```

Так как в лямбда-выражении определена операция сложения параметров, результатом метода будет сумма чисел 10 и 20.

При этом для одного функционального интерфейса мы можем определить множество лямбда-выражений. Например:

```
Operationable operation1 = (int x, int y)-> x + y;  
Operationable operation2 = (int x, int y)-> x - y;  
Operationable operation3 = (int x, int y)-> x * y;
```

```
System.out.println(operation1.calculate(20, 10)); //30  
System.out.println(operation2.calculate(20, 10)); //10  
System.out.println(operation3.calculate(20, 10)); //200
```


Отложенное выполнение



Одним из ключевых моментов в использовании лямбд является отложенное выполнение (deferred execution). То есть мы определяем в одном месте программы лямбда-выражение и затем можем его вызывать при необходимости неопределенное количество раз в различных частях программы. Отложенное выполнение может потребоваться, к примеру, в следующих случаях:

- Выполнение кода отдельном потоке;
- Выполнение одного и того же кода несколько раз;
- Выполнение кода в результате какого-то события;
- Выполнение кода только в том случае, когда он действительно необходим и если он необходим.

Передача параметров в лямбда-выражение



Параметры лямбда-выражения должны соответствовать по типу параметрам метода из функционального интерфейса. При написании самого лямбда-выражения тип параметров писать необязательно, хотя в принципе это можно сделать, например:

```
operation = (int x, int y) -> x + y;
```

Если метод не принимает никаких параметров, то пишутся пустые скобки, например:

```
() -> 30 + 20;
```

Если метод принимает только один параметр, то скобки можно опустить:

```
n -> n * n;
```

Терминальные лямбда-выражения



Выше мы рассмотрели лямбда-выражения, которые возвращают определенное значение. Но также могут быть и терминальные лямбды, которые не возвращают никакого значения. Например:

```
interface Printable{  
    void print(String s);  
}
```

```
public class LambdaApp {
```

```
    public static void main(String[] args) {
```

```
        Printable printer = s->System.out.println(s);
```

```
        printer.print("Hello Java!");
```

```
    }
```

```
}
```

Лямбды и локальные переменные



Лямбда-выражение может использовать переменные, которые объявлены во вне в более общей области видимости - на уровне класса или метода, в котором лямбда-выражение определено. Однако в зависимости от того, как и где определены переменные, могут различаться способы их использования в лямбдах. Рассмотрим первый пример - использования переменных уровня класса:

```
public class LambdaApp {  
  
    static int x = 10;  
    static int y = 20;  
    public static void main(String[] args) {  
  
        Operation op = ()->{  
  
            x=30;  
            return x+y;  
        };  
        System.out.println(op.calculate()); // 50  
        System.out.println(x); // 30 - значение x изменилось  
    }  
}  
  
interface Operation{  
    int calculate();  
}
```

Лямбды и локальные переменные



Переменные `x` и `y` объявлены на уровне класса, и в лямбда-выражении мы их можем получить и даже изменить. Так, в данном случае после выполнения выражения изменяется значение переменной `x`.
Теперь рассмотрим другой пример - локальные переменные на уровне метода:

```
public static void main(String[] args) {  
  
    int n=70;  
    int m=30;  
    Operation op = ()->{  
  
        //n=100; - так нельзя сделать  
        return m+n;  
    };  
    // n=100; - так тоже нельзя  
    System.out.println(op.calculate()); // 100  
}
```


Лямбды и локальные переменные



Локальные переменные уровня метода мы также можем использовать в лямбдах, но изменять их значение нельзя.

Более того, мы не сможем изменить значение переменной, которая используется в лямбда-выражении, вне этого выражения. То есть даже если такая переменная не объявлена как константа, по сути она является константой.

Блоки кода в лямбда-выражениях



Существуют два типа лямбда-выражений: однострочное выражение и блок кода. Примеры однострочных выражений демонстрировались выше.

Блочные выражения обрамляются фигурными скобками. В блочных лямбда-выражениях можно использовать внутренние вложенные блоки, циклы, конструкции if, switch, создавать переменные и т.д. Если блочное лямбда-выражение должно возвращать значение, то явным образом применяется оператор return:

```
Operationable operation = (int x, int y)-> {
```

```
    if (y==0)
        return 0;
    else
        return x/y;
};
```

```
System.out.println(operation.calculate(20, 10)); //2
```

```
System.out.println(operation.calculate(20, 0)); //0
```


Обобщенный функциональный интерфейс



Функциональный интерфейс может быть обобщенным, однако в лямбда-выражении использование обобщений не допускается. В этом случае нам надо типизировать объект интерфейса определенным типом, который потом будет применяться в лямбда-выражении. Например:

```
public class LambdaApp {  
  
    public static void main(String[] args) {  
  
        Operationable<Integer> operation1 = (x, y) -> x + y;  
        Operationable<String> operation2 = (x, y) -> x + y;  
  
        System.out.println(operation1.calculate(20, 10));  
        //30  
        System.out.println(operation2.calculate("20", "10"));  
        //2010  
    }  
}  
  
interface Operationable<T>{  
    T calculate(T x, T y);  
}
```

Работа с коллекциями



Таким образом, при объявлении лямбда-выражения ему уже известно, какой тип параметры будут представлять и какой тип они будут возвращать.

Лямбды как параметры методов

Одним из преимуществ лямбд в java является то, что их можно передавать в качестве параметров в методы. Рассмотрим пример:

```
public class LambdaApp {  
  
    public static void main(String[] args) {  
  
        Expression func = (n) -> n%2==0;  
        int[] nums = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };  
        System.out.println(sum(nums, func)); // 20  
    }  
    private static int sum (int[] numbers, Expression func)  
    {  
        int result = 0;  
        for(int i : numbers)  
        {  
            if (func.isEqual(i))  
                result += i;  
        }  
        return result;  
    }  
}  
interface Expression{  
    boolean isEqual(int n);  
}
```

Работа с коллекциями



Функциональный интерфейс `Expression` определяет метод `isEqual()`, который возвращает `true`, если в отношении числа `n` действует какое-нибудь равенство.

В основном классе программы определяется метод `sum()`, который вычисляет сумму всех элементов массива, соответствующих некоторому условию. А само условие передается через параметр `Expression func`. Причем на момент написания метода `sum` мы можем абсолютно не знать, какое именно условие будет использоваться. Само же условие определяется в виде лямбда-выражения:

```
Expression func = (n) -> n%2==0;
```

То есть в данном случае все числа должны быть четными или остаток от их деления на 2 должен быть равен 0. Затем это лямбда-выражение передается в вызов метода `sum`.

При этом можно не определять переменную интерфейса, а сразу передать в метод лямбда-выражение:

```
int[] nums = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };  
int x = sum(nums, (n) -> n > 5); // сумма чисел, которые больше  
5  
System.out.println(x); // 30
```

Ссылки на метод как параметры методов



Начиная с JDK 8 в Java можно в качестве параметра в метод передавать ссылку на другой метод. В принципе данный способ аналогичен передаче в метод лямбда-выражения.

Ссылка на метод передается в виде имя_класса::
имя_статического_метода (если метод статический)
или объект_класса::имя_метода (если метод
нестатический). Рассмотрим на примере:

```
// функциональный интерфейс
interface Expression{
    boolean isEqual(int n);
}

// класс, в котором определены методы
class ExpressionHelper{

    static boolean isEven(int n){

        return n%2 == 0;
    }

    static boolean isPositive(int n){

        return n > 0;
    }
}

public class LambdaApp {

    public static void main(String[] args) {

        int[] nums = { -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5};
        System.out.println(sum(nums, ExpressionHelper::isEven));

        Expression expr = ExpressionHelper::isPositive;
        System.out.println(sum(nums, expr));
    }

    private static int sum (int[] numbers, Expression func)
    {
        int result = 0;
        for(int i : numbers)
        {
            if (func.isEqual(i))
                result += i;
        }
        return result;
    }
}
```

Ссылки на метод как параметры методов



Здесь также определен функциональный интерфейс `Expression`, который имеет один метод. Кроме того, определен класс `ExpressionHelper`, который содержит два статических метода. В принципе их можно было определить и в основном классе программы, но я вынес их в отдельный класс. В основном классе программы `LambdaApp` определен метод `sum()`, который возвращает сумму элементов массива, соответствующих некоторому условию. Условие передается в виде объекта функционального интерфейса `Expression`.

В методе `main` два раза вызываем метод `sum`, передавая в него один и тот же массив чисел, но разные условия. Первый вызов метода `sum`:

```
System.out.println(sum(nums, ExpressionHelper::isEven));
```


Ссылки на метод как параметры методов



На место второго параметра передается `ExpressionHelper::isEven`, то есть ссылка на статический метод `isEven()` класса `ExpressionHelper`. При этом методы, на которые идет ссылка, должны совпадать по параметрам и результату с методом функционального интерфейса.

При втором вызове метода `sum` отдельно создается объект `Expression`, который затем передается в метод:

```
Expression expr = ExpressionHelper::isPositive;  
System.out.println(sum(nums, expr));
```

Ссылки на метод как параметры методов



Использование ссылок на методы в качестве параметров аналогично использованию лямбда-выражений.

Если нам надо вызвать нестатические методы, то в ссылке вместо имени класса применяется имя объекта этого класса:

```
interface Expression{
    boolean isEqual(int n);
}

class ExpressionHelper{

    boolean isEven(int n){

        return n%2 == 0;
    }
}

public class LambdaApp {

    public static void main(String[] args) {

        int[] nums = { -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5};
        ExpressionHelper exprHelper= new ExpressionHelper();
        System.out.println(sum(nums, exprHelper::isEven)); // 0
    }

    private static int sum (int[] numbers, Expression func)
    {
        int result = 0;
        for(int i : numbers)
        {
            if (func.isEqual(i))
                result += i;
        }
        return result;
    }
}
```


Ссылки на конструкторы



Подобным образом мы можем использовать конструкторы: название_класса::new.

Например:

```
public class LambdaApp {  
  
    public static void main(String[] args) {  
  
        UserBuilder userBuilder = User::new;  
        User user = userBuilder.create("Tom");  
        System.out.println(user.getName());  
    }  
}  
  
interface UserBuilder{  
    User create(String name);  
}  
  
class User{  
  
    private String name;  
    String getName(){  
        return name;  
    }  
  
    User(String n){  
        this.name=n;  
    }  
}
```


Ссылки на конструкторы



При использовании конструкторов методы функциональных интерфейсов должны принимать тот же список параметров, что и конструкторы класса, и должны возвращать объект данного класса.

Лямбды как результат методов



Также метод в Java может возвращать лямбда-выражение. Рассмотрим следующий пример:

```
interface Operation{
    int execute(int x, int y);
}

public class LambdaApp {

    public static void main(String[] args) {

        Operation func = action(1);
        int a = func.execute(6, 5);
        System.out.println(a);           // 11

        int b = action(2).execute(8, 2);
        System.out.println(b);           // 6
    }

    private static Operation action(int number){
        switch(number){
            case 1: return (x, y) -> x + y;
            case 2: return (x, y) -> x - y;
            case 3: return (x, y) -> x * y;
            default: return (x, y) -> 0;
        }
    }
}
```

Лямбды как результат методов



В данном случае определен функциональный интерфейс Operation, в котором метод execute принимает два значения типа int и возвращает значение типа int.

Метод action принимает в качестве параметра число и в зависимости от его значения возвращает то или иное лямбда-выражение. Оно может представлять либо сложение, либо вычитание, либо умножение, либо просто возвращает 0. Стоит учитывать, что формально возвращаемым типом метода action является интерфейс Operation, а возвращаемое лямбда-выражение должно соответствовать этому интерфейсу.

В методе main мы можем вызвать этот метод action. Например, сначала получить его результат - лямбда-выражение, которое присваивается переменной Operation. А затем через метод execute выполнить это лямбда-выражение:

```
Operation func = action(1);  
int a = func.execute(6, 5);  
System.out.println(a);           // 11
```

Либо можно сразу получить и тут же выполнить лямбда-выражение:

```
int b = action(2).execute(8, 2);  
System.out.println(b);           // 6
```

Лямбды как результат методов



В JDK 8 вместе с самой функциональностью лямбда-выражений также было добавлено некоторое количество встроенных функциональных интерфейсов, которые мы можем использовать в различных ситуациях и в различные API в рамках JDK 8. В частности, ряд далее рассматриваемых интерфейсов широко применяется в Stream API. Рассмотрим основные из этих интерфейсов:

- Predicate<T>
- Consumer<T>
- Function<T,R>
- Supplier<T>
- UnaryOperator<T>
- BinaryOperator<T>

Predicate<T>



Функциональный интерфейс Predicate<T> проверяет соблюдение некоторого условия. Если оно соблюдается, то возвращается значение true. В качестве параметра лямбда-выражение принимает объект типа T:

```
public interface Predicate<T> {  
    boolean test(T t);  
}  
  
import java.util.function.Predicate;  
  
public class LambdaApp {  
  
    public static void main(String[] args) {  
  
        Predicate<Integer> isPositive = x -> x > 0;  
  
        System.out.println(isPositive.test(5)); // true  
        System.out.println(isPositive.test(-7)); // false  
    }  
}
```


BinaryOperator<T>



BinaryOperator<T> принимает в качестве параметра два объекта типа T, выполняет над ними бинарную операцию и возвращает ее результат также в виде объекта типа T:

```
public interface BinaryOperator<T> {  
    T apply(T t1, T t2);  
}  
  
import java.util.function.BinaryOperator;  
  
public class LambdaApp {  
  
    public static void main(String[] args) {  
  
        BinaryOperator<Integer> multiply = (x, y) -> x*y;  
  
        System.out.println(multiply.apply(3, 5)); // 15  
        System.out.println(multiply.apply(10, -2)); // -20  
    }  
}
```

UnaryOperator<T>



UnaryOperator<T> принимает в качестве параметра объект типа T, выполняет над ними операции и возвращает результат операций в виде объекта типа T:

```
public interface UnaryOperator<T> {  
    T apply(T t);  
}  
  
import java.util.function.UnaryOperator;  
  
public class LambdaApp {  
  
    public static void main(String[] args) {  
  
        UnaryOperator<Integer> square = x -> x*x;  
        System.out.println(square.apply(5)); // 25  
    }  
}
```

Function<T,R>



Функциональный интерфейс Function<T,R> представляет функцию перехода от объекта типа T к объекту типа R:

```
public interface Function<T, R> {  
    R apply(T t);  
}
```

```
import java.util.function.Function;
```

```
public class LambdaApp {
```

```
    public static void main(String[] args) {
```

```
        Function<Integer, String> convert = x-> String.valueOf(x) + "  
долларов";
```

```
        System.out.println(convert.apply(5)); // 5 долларов
```

```
    }  
}
```


Consumer<T>



Consumer<T> выполняет некоторое действие над объектом типа T, при этом ничего не возвращая:

```
public interface Consumer<T> {  
    void accept(T t);  
}  
  
import java.util.function.Consumer;  
  
public class LambdaApp {  
  
    public static void main(String[] args) {  
  
        Consumer<Integer> printer = x-> System.out.printf("%d долларов \n",  
x);  
        printer.accept(600); // 600 долларов  
    }  
}
```

Supplier<T>



Supplier<T> не принимает никаких аргументов, но должен возвращать объект типа T:

```
public interface Supplier<T> {
    T get();
}

import java.util.Scanner;
import java.util.function.Supplier;

public class LambdaApp {

    public static void main(String[] args) {

        Supplier<User> userFactory = ()->{

            Scanner in = new Scanner(System.in);
            System.out.println("Введите имя: ");
            String name = in.nextLine();
            return new User(name);
        };

        User user1 = userFactory.get();
        User user2 = userFactory.get();

        System.out.println("Имя user1: " + user1.getName());
        System.out.println("Имя user2: " + user2.getName());
    }
}

class User{

    private String name;

    String getName() {
        return name;
    }

    User(String n) {
        this.name=n;
    }
}
```

Stream API



Начиная с JDK 8 в Java появился новый API - Stream API. Его задача - упростить работу с наборами данных, в частности, упростить операции фильтрации, сортировки и другие манипуляции с данными. Вся основная функциональность данного API сосредоточена в пакете `java.util.stream`.

Ключевым понятием в Stream API является поток данных. Применительно к Stream API поток представляет канал передачи данных из источника данных. Причем в качестве источника могут выступать как файлы, так и массивы и коллекции.

Одной из отличительных черт Stream API является применение лямбда-выражений, которые позволяют значительно сократить запись выполняемых действий.

Stream API



Рассмотрим простейший пример. Допустим, у нас есть задача: найти в массиве количество всех чисел, которые больше 0. До JDK 8 мы бы могли написать что-то наподобие следующего:

```
int[] numbers = {-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5};
int count=0;
for(int i:numbers){

    if(i > 0) count++;
}
System.out.println(count);
```

Теперь применим Stream API:

```
import java.util.stream.*;
//.....
long count = IntStream.of(-5, -4, -3, -2, -1, 0, 1, 2, 3, 4,
5).filter(w -> w > 0).count();
System.out.println(count);
```

Stream API



При работе со Stream API важно понимать, что все операции с потоками бывают либо терминальными (terminal), либо промежуточными (intermediate). Промежуточные операции возвращают трансформированный поток. Например, выше в примере метод `filter` принимал поток чисел и возвращал уже преобразованный поток, в котором только числа больше 0. К возвращенному потоку также можно применить ряд промежуточных операций.

Конечные, или терминальные, операции возвращают конкретный результат. Например, в примере выше метод `count()` представляет терминальную операцию и возвращает число. После этого никаких промежуточных операций естественно применять нельзя.

Stream API



Все потоки производят вычисления, в том числе в промежуточных операциях, только тогда, когда к ним применяется терминальная операция. То есть в данном случае применяется отложенное выполнение.

В основе Stream API лежит интерфейс BaseStream. Его полное определение:

```
interface BaseStream<T , S extends BaseStream<T , S>>
```

Здесь параметр T означает тип данных в потоке, а S - тип потока, который наследуется от интерфейса BaseStream.

Stream API



BaseStream определяет базовый функционал для работы с потоками, которые реализуется через его методы:

- **void close():** закрывает поток
- **boolean isParallel():** возвращает true, если поток является параллельным
- **Iterator<T> iterator():** возвращает ссылку на итератор потока
- **Splitter<T> splitter():** возвращает ссылку на сплитератор потока
- **S parallel():** возвращает параллельный поток (параллельные потоки могут задействовать несколько ядер процессора в многоядерных архитектурах)
- **S sequential():** возвращает последовательный поток
- **S unordered():** возвращает неупорядоченный поток

Stream API



От интерфейса BaseStream наследуется ряд интерфейсов, предназначенных для создания конкретных потоков:

- **Stream<T>:** используется для потоков данных, представляющих любой ссылочный тип
- **IntStream:** используется для потоков с типом данных int
- **DoubleStream:** используется для потоков с типом данных double
- **LongStream:** используется для потоков с типом данных long

Stream API



При работе с потоками, которые представляют определенный примитивный тип - double, int, long проще использовать интерфейсы DoubleStream, IntStream, LongStream. Но в большинстве случаев, как правило, работа происходит с более сложными данными, для которых предназначен интерфейс Stream<T>.

Рассмотрим некоторые его методы:

- **boolean allMatch(Predicate<? super T> predicate):** возвращает true, если все элементы потока удовлетворяют условию в предикате. Терминальная операция
- **boolean anyMatch(Predicate<? super T> predicate):** возвращает true, если хоть один элемент потока удовлетворяет условию в предикате. Терминальная операция
- **<R,A> R collect(Collector<? super T,A,R> collector):** добавляет элементы в неизменяемый контейнер с типом R. T представляет тип данных из вызывающего потока, а A - тип данных в контейнере. Терминальная операция
- **long count():** возвращает количество элементов в потоке. Терминальная операция.

Stream API



- **Stream<T> concat(Stream<? extends T> a, Stream<? extends T> b):**
объединяет два потока. Промежуточная операция
- **Stream<T> distinct():** возвращает поток, в котором имеются только уникальные данные с типом T. Промежуточная операция
- **Stream<T> dropWhile(Predicate<? super T> predicate):** пропускает элементы, которые соответствуют условию в predicate, пока не попадется элемент, который не соответствует условию. Выбранные элементы возвращаются в виде потока. Промежуточная операция.
- **Stream<T> filter(Predicate<? super T> predicate):** фильтрует элементы в соответствии с условием в предикате. Промежуточная операция
- **Optional<T> findFirst():** возвращает первый элемент из потока.
Терминальная операция
- **Optional<T> findAny():** возвращает первый попавшийся элемент из потока.
Терминальная операция
- **void forEach(Consumer<? super T> action):** для каждого элемента выполняется действие action. Терминальная операция

Stream API



- **Stream<T> limit(long maxSize):** оставляет в потоке только maxSize элементов. Промежуточная операция
- **Optional<T> max(Comparator<? super T> comparator):** возвращает максимальный элемент из потока. Для сравнения элементов применяется компаратор comparator. Терминальная операция
- **Optional<T> min(Comparator<? super T> comparator):** возвращает минимальный элемент из потока. Для сравнения элементов применяется компаратор comparator. Терминальная операция
- **<R> Stream<R> map(Function<? super T, ? extends R> mapper):** преобразует элементы типа T в элементы типа R и возвращает поток с элементами R. Промежуточная операция
- **<R> Stream<R> flatMap(Function<? super T, ? extends Stream<? extends R>> mapper):** позволяет преобразовать элемент типа T в несколько элементов типа R и возвращает поток с элементами R. Промежуточная операция
- **boolean noneMatch(Predicate<? super T> predicate):** возвращает true, если ни один из элементов в потоке не удовлетворяет условию в предикате. Терминальная операция

Stream API



- **Stream<T> skip(long n):** возвращает поток, в котором отсутствуют первые n элементов. Промежуточная операция.
- **Stream<T> sorted():** возвращает отсортированный поток. Промежуточная операция.
- **Stream<T> sorted(Comparator<? super T> comparator):** возвращает отсортированный в соответствии с компаратором поток. Промежуточная операция.
- **Stream<T> takeWhile(Predicate<? super T> predicate):** выбирает из потока элементы, пока они соответствуют условию в predicate. Выбранные элементы возвращаются в виде потока. Промежуточная операция.
- **Object[] toArray():** возвращает массив из элементов потока. Терминальная операция.

Stream API



Несмотря на то, что все эти операции позволяют взаимодействовать с потоком как неким набором данных наподобие коллекции, важно понимать отличие коллекций от потоков:

Потоки не хранят элементы. Элементы, используемые в потоках, могут храниться в коллекции, либо при необходимости могут быть напрямую сгенерированы.

Операции с потоками не изменяют источника данных. Операции с потоками лишь возвращают новый поток с результатами этих операций.

Для потоков характерно отложенное выполнение. То есть выполнение всех операций с потоком происходит лишь тогда, когда выполняется терминальная операция и возвращается конкретный результат, а не новый поток.

Для создания потока данных можно применять различные методы. В качестве источника потока мы можем использовать коллекции.

Stream API



Так, рассмотрим пример с ArrayList:

```
import java.util.stream.Stream;
import java.util.*;
public class Program {

    public static void main(String[] args) {

        ArrayList<String> cities = new ArrayList<String>();
        Collections.addAll(cities, "Париж", "Лондон",
"Мадрид");
        cities.stream() // получаем поток
            .filter(s->s.length()==6) // применяем фильтрацию
по длине строки
            .forEach(s->System.out.println(s)); // выводим
отфильтрованные строки на консоль
    }
}
```

Здесь с помощью вызова `cities.stream()` получаем поток, который использует данные из списка `cities`. С помощью каждой промежуточной операции, которая применяется к потоку, мы также можем получить поток с учетом модификаций.

Stream API



Важно, что после использования терминальных операций другие терминальные или промежуточные операции к этому же потоку не могут быть применены, поток уже употреблен. Например, в следующем случае мы получим ошибку:

```
citiesStream.forEach(s->System.out.println(s)); //  
терминальная операция употребляет поток  
long number = citiesStream.count(); // здесь ошибка, так как  
поток уже употреблен  
System.out.println(number);  
citiesStream = citiesStream.filter(s->s.length()>5); // тоже  
нельзя, так как поток уже употреблен
```

Фактически жизненный цикл потока проходит следующие три стадии:

- Создание потока;
- Применение к потоку ряда промежуточных операций;
- Применение к потоку терминальной операции и получение результата.

Stream API



Кроме вышерассмотренных методов мы можем использовать еще ряд способов для создания потока данных. Один из таких способов представляет метод `Arrays.stream(T[] array)`, который создает поток данных из массива:

```
Stream<String> citiesStream = Arrays.stream(new
String[]{"Париж", "Лондон", "Мадрид"}) ;
citiesStream.forEach(s->System.out.println(s)); // выводим все
элементы массива
```

Для создания потоков `IntStream`, `DoubleStream`, `LongStream` можно использовать соответствующие перегруженные версии этого метода:

```
IntStream intStream = Arrays.stream(new int[]{1, 2, 4, 5, 7});
intStream.forEach(i->System.out.println(i));
```

```
LongStream longStream = Arrays.stream(new
long[]{100, 250, 400, 5843787, 237});
longStream.forEach(l->System.out.println(l));
```

```
DoubleStream doubleStream = Arrays.stream(new double[] {3.4,
6.7, 9.5, 8.2345, 121});
doubleStream.forEach(d->System.out.println(d));
```


Stream API



И еще один способ создания потока представляет статический метод `of(T..values)` класса `Stream`:

```
Stream<String> citiesStream = Stream.of("Париж", "Лондон",  
"Мадрид");  
citiesStream.forEach(s->System.out.println(s));
```

```
// можно передать массив  
String[] cities = {"Париж", "Лондон", "Мадрид"};  
Stream<String> citiesStream2 = Stream.of(cities);
```

```
IntStream intStream = IntStream.of(1,2,4,5,7);  
intStream.forEach(i->System.out.println(i));
```

```
LongStream longStream =  
LongStream.of(100,250,400,5843787,237);  
longStream.forEach(l->System.out.println(l));
```

```
DoubleStream doubleStream = DoubleStream.of(3.4, 6.7, 9.5,  
8.2345, 121);  
doubleStream.forEach(d->System.out.println(d));
```

Перебор элементов. Метод `forEach`



Для перебора элементов потока применяется метод `forEach()`, который представляет терминальную операцию. В качестве параметра он принимает объект `Consumer<? super String>`, который представляет действие, выполняемое для каждого элемента набора. Например:

```
Stream<String> citiesStream = Stream.of("Париж", "Лондон",  
"Мадрид", "Берлин", "Брюссель");  
citiesStream.forEach(s -> System.out.println(s));
```

Фактически это будет аналогично перебору всех элементов в цикле `for` и выполнению с ними действия, а именно, вывод на консоль.

Кстати, мы можем сократить в данном случае применение метода `forEach` следующим образом:

```
Stream<String> citiesStream = Stream.of("Париж", "Лондон",  
"Мадрид", "Берлин", "Брюссель");  
citiesStream.forEach(System.out::println);
```

Фактически здесь передается ссылка на статический метод, который выводит строку на консоль.

Фильтрация. Метод filter



Для фильтрации элементов в потоке применяется метод `filter()`, который представляет промежуточную операцию. Он принимает в качестве параметра некоторое условие в виде объекта `Predicate<T>` и возвращает новый поток из элементов, которые удовлетворяют этому условию:

```
Stream<String> citiesStream = Stream.of("Париж", "Лондон",  
"Мадрид", "Берлин", "Брюссель");  
citiesStream.filter(s->s.length()==6).forEach(s->System.out.pr  
intln(s));  
//Лондон  
//Мадрид  
//Берлин
```

Фильтрация. Метод filter



Рассмотрим еще один пример фильтрации с более сложными данными.

Допустим, у нас есть следующий класс Phone:

```
class Phone{

    private String name;
    private int price;

    public Phone(String name, int price){
        this.name=name;
        this.price=price;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getPrice() {
        return price;
    }

    public void setPrice(int price) {
        this.price = price;
    }
}
```

Фильтрация. Метод filter



Отфильтруем набор телефонов по цене:

```
Stream<Phone> phoneStream = Stream.of(new Phone("iPhone 6 S",  
54000), new Phone("Lumia 950", 45000),  
new Phone("Samsung Galaxy S 6", 40000));
```

```
phoneStream.filter(p->p.getPrice()<50000).forEach(p->  
System.out.println(p.getName()));
```


Отображение. Метод map



Отображение или маппинг позволяет задать функцию преобразования одного объекта в другой, то есть получить из элемента одного типа элемент другого типа. Для отображения используется метод map, который имеет следующее определение:

```
<R> Stream<R> map(Function<? super T, ? extends R> mapper)
```

Передаваемая в метод map функция задает преобразование от объектов типа T к типу R. И в результате возвращается новый поток с преобразованными объектами.

Возьмем вышеопределенный класс телефонов и выполним преобразование от типа Phone к типу String:

```
Stream<Phone> phoneStream = Stream.of(new Phone("iPhone 6 S",  
54000), new Phone("Lumia 950", 45000),  
new Phone("Samsung Galaxy S 6", 40000));
```

```
phoneStream  
    .map(p-> p.getName()) // помещаем в поток только названия  
телефонов  
    .forEach(s->System.out.println(s));  
//iPhone 6 S  
//Lumia 950  
//Samsung Galaxy S 6
```

Отображение. Метод map



Операция `map(p-> p.getName())` помещает в новый поток только названия телефонов. В итоге на консоли будут только названия.

Для преобразования объектов в типы `Integer`, `Long`, `Double` определены специальные методы `mapToInt()`, `mapToLong()` и `mapToDouble()` соответственно.

Коллекции, на основе которых нередко создаются потоки, уже имеют специальные методы для сортировки содержимого. Однако класс `Stream` также включает возможность сортировки. Такую сортировку мы можем задействовать, когда у нас идет набор промежуточных операций с потоком, которые создают новые наборы данных, и нам надо эти наборы отсортировать.

Отображение. Метод map



Для простой сортировки по возрастанию применяется метод sorted():

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Program {

    public static void main(String[] args) {

        List<String> phones = new ArrayList<String>();
        Collections.addAll(phones, "iPhone X", "Nokia 9", "Huawei Nexus 6P", "Samsung Galaxy
S8", "LG G6", "Xiaomi MI6", "ASUS Zenfone 3", "Sony Xperia Z5", "Meizu Pro 6", "Pixel 2");

        phones.stream()
            .filter(p -> p.length() < 12)
            .sorted() // сортировка по возрастанию
            .forEach(s -> System.out.println(s));
    }
}

//LG G6
//Meizu Pro 6
//Nokia 9
//Pixel 2
//Xiaomi MI6
//iPhone X
```


Отображение. Метод `map`



Однако данный метод не всегда подходит. Уже по консольному выводу мы видим, что метод сортирует объекты по возрастанию, но при этом заглавные и строчные буквы рассматриваются отдельно.

Кроме того, данный метод подходит только для сортировки тех объектов, которые реализуют интерфейс `Comparable`.

Если же у нас классы объектов не реализуют этот интерфейс или мы хотим создать какую-то свою логику сортировки, то мы можем использовать другую версию метода `sorted()`, которая в качестве параметра принимает компаратор.

Отображение. Метод map



Например, пусть у нас есть следующий класс Phone:

```
class Phone{  
  
    private String name;  
    private String company;  
    private int price;  
  
    public Phone(String name, String comp, int price){  
        this.name=name;  
        this.company=comp;  
        this.price = price;  
    }  
  
    public String getName() { return name; }  
    public int getPrice() { return price; }  
    public String getCompany() { return company; }  
}
```

Отображение. Метод map



Отсортируем поток объектов Phone:

```
import java.util.Comparator;
import java.util.stream.Stream;

public class Program {

    public static void main(String[] args) {

        Stream<Phone> phoneStream = Stream.of(new Phone("iPhone X", "Apple", 600),
            new Phone("Pixel 2", "Google", 500),
            new Phone("iPhone 8", "Apple", 450),
            new Phone("Nokia 9", "HMD Global", 150),
            new Phone("Galaxy S9", "Samsung", 300));

        phoneStream.sorted(new PhoneComparator())
            .forEach(p->System.out.printf("%s (%s) - %d \n",
                p.getName(), p.getCompany(), p.getPrice()));

    }
}

class PhoneComparator implements Comparator<Phone> {

    public int compare(Phone a, Phone b){

        return a.getName().toUpperCase().compareTo(b.getName().toUpperCase());

    }
}

//Galaxy S9 (Samsung) - 300
//iPhone 8 (Apple) - 450
//iPhone X (Apple) - 600
//Nokia 9 (HMD Global) - 150
//Pixel 2 (Google) - 500
```

Здесь определен класс компаратора

PhoneComparator, который сортирует объекты по полю name.

Методы skip и limit



Метод `skip(long n)` используется для пропуска `n` элементов. Этот метод возвращает новый поток, в котором пропущены первые `n` элементов.

Метод `limit(long n)` применяется для выборки первых `n` элементов потоков. Этот метод также возвращает модифицированный поток, в котором не более `n` элементов.

Зачастую эта пара методов используется вместе для создания эффекта постраничной навигации. Рассмотрим, как их применять:

```
Stream<String> phoneStream = Stream.of("iPhone 6 S", "Lumia  
950", "Samsung Galaxy S 6", "LG G 4", "Nexus 7");
```

```
phoneStream.skip(1)  
    .limit(2)  
    .forEach(s->System.out.println(s));  
//Lumia 950  
//Samsung Galaxy S 6
```

Методы `skip` и `limit`



В данном случае метод `skip` пропускает один первый элемент, а метод `limit` выбирает два следующих элемента.

Вполне может быть, что метод `skip` может принимать в качестве параметра число большее, чем количество элементов в потоке. В этом случае будут пропущены все элементы, а в результирующем потоке будет 0 элементов.

И если в метод `limit` передается число, большее, чем количество элементов, то просто выбираются все элементы потока.

Операции сведения



Операции сведения представляют терминальные операции, которые возвращают некоторое значение - результат операции. В Stream API есть ряд операций сведения.

count

Метод count() возвращает количество элементов в потоке данных:

```
import java.util.stream.Stream;
import java.util.Optional;
import java.util.*;
public class Program {

    public static void main(String[] args) {

        ArrayList<String> names = new ArrayList<String>();
        names.addAll(Arrays.asList(new String[]{"Tom", "Sam", "Bob",
"Alice"}));
        System.out.println(names.stream().count()); // 4

        // количество элементов с длиной не больше 3 символов

        System.out.println(names.stream().filter(n->n.length()<=3).count());
        // 3
    }
}
```

Операции сведения



findFirst и **findAny**

Метод `findFirst()` извлекает из потока первый элемент, а `findAny()` извлекает случайный объект из потока (нередко так же первый):

```
ArrayList<String> names = new ArrayList<String>();  
names.addAll(Arrays.asList(new String[]{"Tom", "Sam", "Bob",  
"Alice"}));
```

```
Optional<String> first = names.stream().findFirst();  
System.out.println(first.get()); // Tom
```

```
Optional<String> any = names.stream().findAny();  
System.out.println(first.get()); // Tom
```

Операции сведения



allMatch, anyMatch, noneMatch

Еще одна группа операций сведения возвращает логическое значение true или false:

- **boolean allMatch(Predicate<? super T> predicate):** возвращает true, если все элементы потока удовлетворяют условию в предикате;
- **boolean anyMatch(Predicate<? super T> predicate):** возвращает true, если хоть один элемент потока удовлетворяет условию в предикате;
- **boolean noneMatch(Predicate<? super T> predicate):** возвращает true, если ни один из элементов в потоке не удовлетворяет условию в предикате.

Операции сведения



allMatch, anyMatch, noneMatch

Пример использования функций:

```
import java.util.stream.Stream;
import java.util.Optional;
import java.util.ArrayList;
import java.util.Arrays;
public class Program {

    public static void main(String[] args) {

        ArrayList<String> names = new ArrayList<String>();
        names.addAll(Arrays.asList(new String[]{"Tom", "Sam", "Bob", "Alice"}));

        // есть ли в потоке строка, длина которой больше 3
        boolean any = names.stream().anyMatch(s->s.length()>3);
        System.out.println(any);    // true

        // все ли строки имеют длину в 3 символа
        boolean all = names.stream().allMatch(s->s.length()==3);
        System.out.println(all);    // false

        // НЕТ ЛИ в потоке строки "Bill". Если нет, то true, если есть, то false
        boolean none = names.stream().noneMatch(s->s=="Bill");
        System.out.println(none);    // true
    }
}
```

Операции сведения



min и max

Методы `min()` и `max()` возвращают соответственно минимальное и максимальное значение. Поскольку данные в потоке могут представлять различные типы, в том числе сложные классы, то в качестве параметра в эти методы передается объект интерфейса `Comparator`, который указывает, как сравнивать объекты.

Оба метода возвращают элемент потока (минимальный или максимальный), обернутый в объект `Optional`.

Например, найдем минимальное и максимальное число в числовом потоке:

```
import java.util.stream.Stream;
import java.util.Optional;
import java.util.ArrayList;
import java.util.Arrays;
public class Program {

    public static void main(String[] args) {

        ArrayList<Integer> numbers = new ArrayList<Integer>();
        numbers.addAll(Arrays.asList(new Integer[]{1,2,3,4,5,6,7,8,9}));

        Optional<Integer> min = numbers.stream().min(Integer::compare);
        Optional<Integer> max = numbers.stream().max(Integer::compare);
        System.out.println(min.get()); // 1
        System.out.println(max.get()); // 9
    }
}
```

Операции сведения



Интерфейс **Comparator** - это функциональный интерфейс, который определяет один метод `compare`, принимающий два сравниваемых объекта и возвращающий число (если первый объект больше, возвращается положительное число, иначе возвращается отрицательное число).

Поэтому вместо конкретной реализации компаратора мы можем передать лямбда-выражение или метод, который соответствует методу `compare` интерфейса `Comparator`. Поскольку сравниваются числа, то в метод передается в качестве компаратора статический метод `Integer.compare()`.

При этом методы `min` и `max` возвращают именно `Optional`, и чтобы получить непосредственно результат операции из `Optional`, необходимо вызвать метод `get()`.

Метод collect



Большинство операций класса Stream, которые модифицируют набор данных, возвращают этот набор в виде потока. Однако бывают ситуации, когда хотелось бы получить данные не в виде потока, а в виде обычной коллекции, например, ArrayList или HashSet. И для этого у класса Stream определен метод collect. Первая версия метода принимает в качестве параметра функцию преобразования к коллекции:

```
<R,A> R collect(Collector<? super T,A,R> collector)
```

Параметр R представляет тип результата метода, параметр T - тип элемента в потоке, а параметр A - тип промежуточных накапливаемых данных. В итоге параметр collector представляет функцию преобразования потока в коллекцию.

Метод collect



Эта функция представляет объект Collector, который определен в пакете `java.util.stream`. Мы можем написать свою реализацию функции, однако Java уже предоставляет ряд встроенных функций, определенных в классе `Collectors`:

- **toList():** преобразование к типу `List`;
- **toSet():** преобразование к типу `Set`;
- **toMap():** преобразование к типу `Map`.

Метод collect



Например, преобразуем набор в потоке в список:

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.stream.Collectors;

public class Program {

    public static void main(String[] args) {

        List<String> phones = new ArrayList<String>();
        Collections.addAll(phones, "iPhone 8", "HTC U12", "Huawei Nexus 6P",
            "Samsung Galaxy S9", "LG G6", "Xiaomi MI6", "ASUS Zenfone 2",
            "Sony Xperia Z5", "Meizu Pro 6", "Lenovo S850");

        List<String> filteredPhones = phones.stream()
            .filter(s->s.length()<10)
            .collect(Collectors.toList());

        for(String s : filteredPhones){
            System.out.println(s);
        }
    }
}
```

Метод collect



Использование метода toSet() аналогично.

```
Set<String> filteredPhones = phones.stream()
    .filter(s->s.length()<10)
    .collect(Collectors.toSet());
```

Для применения метода toMap() надо задать ключ и значение. Например, пусть у нас есть следующая модель:

```
class Phone{

    private String name;
    private int price;

    public Phone(String name, int price){
        this.name=name;
        this.price=price;
    }

    public String getName() {
        return name;
    }

    public int getPrice() {
        return price;
    }
}
```

Метод collect



Теперь применим метод toMap():

```
import java.util.Map;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class Program {

    public static void main(String[] args) {

        Stream<Phone> phoneStream = Stream.of(new Phone("iPhone 8", 54000),
            new Phone("Nokia 9", 45000),
            new Phone("Samsung Galaxy S9", 40000),
            new Phone("LG G6", 32000));

        Map<String, Integer> phones = phoneStream
            .collect(Collectors.toMap(p->p.getName(), t->t.getPrice()));

        phones.forEach((k,v)->System.out.println(k + " " + v));
    }
}

class Phone{

    private String name;
    private int price;

    public Phone(String name, int price){
        this.name=name;
        this.price=price;
    }

    public String getName() { return name; }
    public int getPrice() { return price; }
}
```


Метод collect



Лямбда-выражение `p->p.getName()` получает значение для ключа элемента, а `t->t.getPrice()` - извлекает значение элемента.

Если нам надо создать какой-то определенный тип коллекции, например, `HashSet`, то мы можем использовать специальные функции, которые определены в классах-коллекций. Например, получим объект `HashSet`:

```
import java.util.HashSet;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class Program {

    public static void main(String[] args) {

        Stream<String> phones = Stream.of("iPhone 8", "HTC U12",
            "Huawei Nexus 6P", "Samsung Galaxy S9", "LG G6", "Xiaomi MI6",
            "ASUS Zenfone 2", "Sony Xperia Z5", "Meizu Pro 6", "Lenovo S850");

        HashSet<String> filteredPhones = phones.filter(s->s.length()<12).
            collect(Collectors.toCollection(HashSet::new));

        filteredPhones.forEach(s->System.out.println(s));
    }
}
```

Метод collect



Выражение `HashSet::new` представляет функцию создания коллекции. Аналогичным образом можно получать другие коллекции, например, `ArrayList`:

```
ArrayList<String> result = phones.collect(Collectors.toCollection(ArrayList::new));
```

Вторая форма метода `collect` имеет три параметра:

```
<R> R collect(Supplier<R> supplier, BiConsumer<R,? super T> accumulator, BiConsumer<R,R> combiner)
```

- `supplier`: создает объект коллекции;
- `accumulator`: добавляет элемент в коллекцию;
- `combiner`: бинарная функция, которая объединяет два объекта.

Метод collect



Применим эту версию метода collect:

```
import java.util.ArrayList;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class Program {

    public static void main(String[] args) {

        Stream<String> phones = Stream.of("iPhone 8", "HTC U12",
"Huawei Nexus 6P", "Samsung Galaxy S9", "LG G6", "Xiaomi MI6",
"ASUS Zenfone 2", "Sony Xperia Z5", "Meizu Pro 6", "Lenovo S850");

        ArrayList<String> filteredPhones = phones.filter(s -> s.length() < 12)
            .collect(
                () -> new ArrayList<String>(), // создаем ArrayList
                (list, item) -> list.add(item), // добавляем в список элемент
                (list1, list2) -> list1.addAll(list2)); // добавляем в список другой список

        filteredPhones.forEach(s -> System.out.println(s));
    }
}
```

ПРАКТИЧЕСКОЕ ЗАДАНИЕ



Написать программу(-ы), позволяющую(-ие) выполнить следующее:

1. Для любого набора случайно-сгенерированных чисел нужно определить количество чётных чисел.

2. Задана коллекция, состоящая из строк: «Highload», «High», «Load», «Highload». Нужно с ней выполнить следующие манипуляции:

2.1. Посчитать, сколько раз объект «High» встречается в коллекции;

2.2. Определить, какой элемент в коллекции находится на первом месте.

Если мы получили пустую коллекцию, то пусть возвращается 0;

2.3. Необходимо вернуть последний элемент, если получили пустую коллекцию, то пусть возвращается 0;

3. Задана коллекция, содержащая элементы "f10", "f15", "f2", "f4", "f4".

Необходимо отсортировать строки в алфавитном порядке и добавить их в массив;

ПРАКТИЧЕСКОЕ ЗАДАНИЕ



4. Создай класс со следующим содержимым:

```
Collection<Student> students = Arrays.asList(  
    new Student("Митрий", 17, Gender.MAN),  
    new Student("Максим", 20, Gender.MAN),  
    new Student("Екатерина", 20, Gender.WOMAN),  
    new Student("Михаил", 28, Gender.MAN)  
);  
  
private enum Gender {  
    MAN,  
    WOMAN  
}  
  
private static class Student {  
    private final String name;  
    private final Integer age;  
    private final Sex gender;  
    public Student(String name, Integer age, Gender gender) {  
        this.name = name;  
        this.age = age;  
        this.gender = gender;  
    }  
    public String getName() {  
        return name;  
    }  
    public Integer getAge() {  
        return age;  
    }  
    public Gender getGender() {  
        return gender;  
    }  
    @Override  
    public String toString() {  
        return "{" +  
            "name='" + name + '\'' +  
            ", age=" + age +  
            ", gender=" + gender +  
            "'}";  
    }  
    @Override  
    public boolean equals(Object o) {  
        if (this == o) return true;  
        if (!(o instanceof Student)) return false;  
        Student student = (Student) o;  
        return Objects.equals(name, student.name) &&  
            Objects.equals(age, student.age) &&  
            Objects.equals(gender, student.gender);  
    }  
    @Override  
    public int hashCode() {  
        return Objects.hash(name, age, gender);  
    }  
}
```


ПРАКТИЧЕСКОЕ ЗАДАНИЕ



- 4.1.** Необходимо узнать средний возраст студентов мужского пола;
- 4.2.** Кому из студентов грозит получение повестки в этом году при условии, что призывной возраст установлен в диапазоне от 18 до 27 лет;
- 5.** Нужно написать программу, которая будет принимать от пользователя ввод различных логинов. Как только пользователь введет пустую строку - программа должна прекратить приём данных от пользователя и вывести в консоль логины, начинающиеся на букву f (строчную).